

15/9/2026

Window functions in SQL

- Window functions allow performing calculations across a set of rows that are related to the current row, without collapsing the result in to single value
- used for aggregating, ranking

'OVER' clause defines the window of rows for the calculation.

→ partition: divides the data in to groups

→ order by: specify the order of rows within each group.

Window functions:

Aggregate	Ranking	Value
AVG()	Row-Numbers()	Lag()
MAX()	Rank()	Lead()
MIN()	Dense-rank()	first()
SUM()	Percent-rank()	last()
COUNT()	NTile()	NTILE()

Syntax:

Select column_name1,

window_function(column_name2)

over ([Partition by column_name3]

[order by column_name4]) as

New_column

from table_name;

Window-function: function used for calculation/ranking
Column-name 1: columns to display
Column-name 2: function column
Column-name 3: Partition by column
Column-name 4: order by column
new-column: Result Alias
table-name: Source table.

Aggregate functions: calculate aggregate over a window of rows while retaining individual rows.

Sum(): Sums values within a window.

Avg(): Calculate the average value within a window

count(): counts rows within a window

max(): returns the maximum value in the window

min(): return the minimum value in the window

eg) `sum()` | `select name, age, department, salary,`
 `Avg(salary) over (`
 `Partition by department`
 `) AS avg-salary`
 `from employee;`

Ranking window functions.

These provide rankings of rows within a partition based on specific criteria.

Rank(): Assigns values to rows, skipping ranks for duplicate

`select name, department, salary,`

`Rank() over (`

`Partition by department`

`order by salary DESC`

`) AS emp-rank from employee;`

Output Name	Department	Salary	emp-rank
Andrew	finance	50000	1
Brian	finance	50000	1
Charles	finance	20000	3
Daniel	Sales	30000	1
Ethan	Sales	20000	2

Annotations: Finance is part 1, Sales is part 2. Rank 1 for 50000, Rank 2 for 20000 in each department. Rank 3 for 20000 in finance.

Dense rank() : gives same rank to rows with equal values, it then continues with the next number without skipping.

Output

Name	Department	Salary	emp_dense_rank
Andrew	finance	50000	1
Brian	finance	50000	1
Charles	finance	20000	2
Daniel	Sales	30000	1
Ethan	Sales	20000	2

Annotations: Dense rank. Rank 1 for 50000, Rank 2 for 20000 in each department. Rank 3 for 20000 in finance is skipped.

Row-number : gives unique number to each row in the result set, increment by 1 for every row, even if values are the same.

Output :

Name	Department	Salary	emp_row_no
Andrew	finance	50000	1
Brian	finance	50000	2
Charles	finance	20000	3
Daniel	Sales	30000	1
Ethan	Sales	20000	2

Part 1

Part 2

Percent Rank() shows where a row stands compared to others in the same group

$$\text{Percent_Rank}() = (\text{rank} - 1) / (\text{Total rows in partition} - 1)$$

output :

Name	Department	Salary	Emp-Percent rank
Andrew	finance	50000	0
Brian	finance	50000	0
Charles	finance	20000	1
Daniel	Sales	30000	0
Ethan	Sales	20000	1

Handwritten calculations for Percent Rank:

- For finance department: $\frac{0}{3} = 0$ for Andrew and Brian, $\frac{1}{3} = \frac{1}{3}$ for Charles.
- For Sales department: $\frac{0}{2} = 0$ for Daniel, $\frac{1}{2} = \frac{1}{2}$ for Ethan.